UA-GNN Experimental Correction Plan
Implementation status review

Review date: 10 October 2026. **Repository:** saisidd007/UA_GNN_Traffic_Forecasting, main commit b6ec083. This review was added to the supplied five-page plan; the original plan follows unchanged.

Scope of the work: The original training and evaluation scripts remain in the repository. The fixes and reruns described below use a new `corrected_experiment.py` path. Its model is not an exact reproduction of the published architecture, so its scores must not replace paper numbers.

Status key

DONE: implemented and exercised in the new path. **PARTIAL:** some deliverables were completed, others were not. **NOT DONE:** remains to be implemented or tested. **UNSUPPORTED:** the plan asserts more than the inspected code establishes. **NOT APPLICABLE:** organisational advice, not a software correction.

New single-seed test results

Dataset	Model MAE / RMSE	Persistence MAE / RMSE	95% coverage / width
METR-LA	4.043 / 7.502 mph	4.182 / 8.111 mph	96.7% / 32.79 mph
PEMS-BAY	2.135 / 4.572 mph	2.228 / 5.225 mph	97.5% / 15.07 mph

Run protocol and limits

Both runs used CPU, seed 42, a 50-epoch maximum, 8,192 newly sampled training windows per epoch, full chronological validation and test splits, and 30 Monte Carlo dropout passes for the final test. Validation selected checkpoints at epoch 49 (METR-LA) and 48 (PEMS-BAY). No five-seed summary, ablations, or revised manuscript was produced.

The METR-LA checkout has no original adjacency file, so the new run used an eight-nearest-neighbour geographic proxy. The wide intervals and coverage above 95% do not establish good calibration. Zero values were treated as missing sentinels based on file patterns; that convention still needs source-backed verification.

The plan names `Chakri745/UA_GNN_Traffic_Forecasting`; the requested checkout and pushed commit are in `saisidd007/UA_GNN_Traffic_Forecasting`.

Status by plan section: 1–5

1. Data preprocessing — PARTIAL. The new path splits forecast targets chronologically, fits per-sensor scaling only on training timestamps, uses causal forward filling, and excludes unobserved targets from losses and metrics. The legacy loaders were not changed. Zero-as-missing was inferred from data patterns, not proven from source documentation. A formal leakage report with all requested timestamps/counts was not produced.

2. Temporal architecture — PARTIAL. The new model aggregates graph neighbours at each historical time step, then applies convolution and LSTM along the 12-step time axis. A forward and shape test passes. The original architecture was not repaired in place; no matching paper diagram or complete tensor-shape table was delivered.

3. Validation/testing — DONE in the new path; one claim UNSUPPORTED. The new path saves checkpoints by validation MSE and evaluates test only after training. The old METR-LA script printed test metrics every epoch, which risks test peeking, but its checkpoint and early-stopping conditions already used validation loss. The plan should not claim that old checkpoints were directly chosen by test score.

4. MC dropout/batch normalisation — DONE in the new path only. MC inference calls `eval()` and enables the dropout module selectively; it uses 30 passes. The legacy evaluation scripts still call `train()` during MC sampling. The new model contains no batch-normalisation layer, so it is not a direct legacy BN regression test.

5. Uncertainty/calibration — PARTIAL. Predictive variance combines the population variance of MC means and the mean positive aleatoric variance. Metrics include MAE/RMSE, 50/80/90/95% coverage, mean interval width, and horizon-specific MAE. The runs did not generate the requested calibration plots, horizon-specific interval tables, or CRPS. Wide intervals indicate the uncertainty estimate needs more scrutiny.

Status by plan section: 6–11

6. Graph and diffusion — NOT DONE as requested. PEMS-BAY uses its supplied adjacency matrix and checks sensor order. METR-LA uses a geographic proxy because its original graph is absent. No diffusion-order/direction audit, hand-constructed graph test, or matched no-diffusion ablation was completed. The unusually large published ablation change is a reason to investigate, not proof of an implementation error.

7. Core experiments — PARTIAL. Both datasets were rerun once (seed 42) with 50 epochs and saved checkpoints, logs, and metrics. A persistence baseline was run on the same test periods. Training used 8,192 sampled windows per epoch, not every training window per epoch. No five-seed mean/standard deviation, key ablations, or reproduced published baselines were produced.

8. Sensor-failure experiments — NOT DONE. Random, persistent, clustered, and contiguous test-time corruption comparisons were not run. The plan describes useful future evidence, not a demonstrated defect in the current code.

9. Additional metrics — PARTIAL. Horizon-specific MAE, interval coverage and width, and a persistence comparison were saved. No CRPS/interval score, paired confidence intervals, latency, throughput, or peak-memory study was performed.

10. Two-student division of work — NOT APPLICABLE. This is a suggested staffing plan, not a technical requirement or code correction.

11. Final checklist/paper consistency — NOT COMPLETE. The new results do not validate the manuscript claims. Original model code and paper artifacts remain, and the missing graph, multi-seed, ablation, robustness, and manuscript updates prevent treating the checklist as passed.

What should not be inferred: Five seeds and extra robustness analyses are reasonable research recommendations, not proof that existing code is wrong. Neither the extreme no-diffusion figure nor the zero-value convention can be settled by assertion alone. The new experiment is auditable but not a controlled one-change-at-a-time replication of the published UA-GNN model.

UA-GNN Traffic Forecasting

Experimental Correction and Validation Plan

Repository: https://github.com/Chakri745/UA_GNN_Traffic_Forecasting

Purpose: Student-facing instructions to correct the experimental pipeline, verify model behaviour, rerun core experiments and strengthen the paper's evidence.

Guiding rule: Preserve original code and results, record every change, and do not overwrite old outputs. Report corrected results even if less favourable.

Execution order

Correct data/model issues first; validate uncertainty and evaluation next; then run repeated-seed, ablation and robustness experiments. Do not spend the full experimental budget repeating a pipeline before checking its correctness.

1. Correct data preprocessing and eliminate leakage

Priority: Essential
Why: The loader appears to fit preprocessing before chronological splitting; backward filling may use later observations to fill earlier missing values.

Files to inspect: Dataset loader/preprocessing module, imputation functions and split creation.

1. Split raw time series chronologically into training, validation and test periods using the intended proportions.

2. Fit scaler and any learned imputer on training data only; apply the fitted transformations to validation/test without refitting.

3. Remove backward filling when it uses future observations unavailable at forecasting time. Use a documented causal alternative or an observation-mask-aware method.

4. Ensure sliding input windows and forecast targets do not cross boundaries in a way that exposes future targets to training.

5. Check how zero values and missing readings are represented; do not conflate genuine low speed with missingness.

Deliverables: Revised preprocessing code, split timestamps, missing-value counts, scaler-fit details and leakage-check report.

Acceptance test: No validation/test values fit preprocessing; no future observation imputes a past input; training examples contain no test-period targets.

2. Correct the temporal model architecture

Priority: Essential
Why: Temporal convolution and LSTM operations may act on the sensor/node dimension instead of the 12-step historical-time dimension. Confirm from tensor shapes before modifying code.

Files to inspect: Model definition containing temporal convolution, LSTM and spatial/diffusion operations.

1. Print and document tensor shape after every major operation, identifying batch, sensor, historical time-step and feature/channel dimensions.

2. Ensure temporal convolution operates along the 12 historical time steps, not across sensors.

3. Ensure the LSTM sequence axis represents historical time; document reshaping and permutation operations.

4. Ensure graph diffusion or graph attention operates along the sensor-graph dimension.

5. Verify output shape is batch × sensors × forecast horizon, with the intended uncertainty output.

Deliverables: Corrected code, tensor-shape table, matching model diagram and a successful forward-pass test.

Acceptance test: Operations match the mathematical formulation; do not merely reshape tensors to make code run.

3. Correct validation, testing and checkpoint selection

Priority: Essential
Why: The training loop appears to evaluate test performance repeatedly, risking decisions influenced by held-out test results.

Files to inspect: Training loop and evaluation/checkpoint-selection functions.

1. Use training data to update model parameters.

2. Use validation loss for checkpoint selection, hyperparameter tuning and early stopping.

3. Do not inspect test metrics every epoch; evaluate test only after the configuration and checkpoint procedure are finalised.

4. Keep test data untouched during hyperparameter selection.

5. Apply consistent split and evaluation rules to PEMS-BAY and METR-LA.

Deliverables: Training logs, selected checkpoint, validation curves and one final test report per dataset.

Acceptance test: Test performance does not determine early stopping, checkpoint selection or hyperparameter decisions.

4. Fix Monte Carlo dropout and batch normalisation

Priority: Essential
Why: Calling model.train() during MC inference can also put batch-normalisation layers into training mode.

Files to inspect: Model-evaluation and Monte Carlo sampling functions.

1. Put the model into evaluation mode before inference.

2. Enable dropout layers selectively for MC sampling while keeping batch-normalisation layers in evaluation mode.

3. Start with 30–50 stochastic passes for debugging and check stability as the pass count increases.

4. Confirm deterministic evaluation is reproducible and MC predictions vary when dropout is enabled.

5. Record the number of sampling passes and dropout configuration.

Deliverables: Tested inference function, a test demonstrating intended dropout behaviour and example stochastic predictions.

5. Correct uncertainty estimation and calibration

Priority: Essential
Why: The calibration script appears to use MC prediction spread alone for intervals, potentially omitting predicted aleatoric variance.

Files to inspect: Variance head, uncertainty loss, MC prediction function and calibration/evaluation script.

1. Let $\mu_m(x)$ be the predicted mean and $\sigma^2_m(x)$ the predicted aleatoric variance from MC pass m. Predictive mean is the average of $\mu_m(x)$.

2. Epistemic variance is the average squared deviation of $\mu_m(x)$ from the predictive mean.

3. Aleatoric variance is the average of $\sigma^2_m(x)$, after conversion from log variance if required.

4. Combine predictive variance as epistemic variance + aleatoric variance. Use a consistent variance convention and denominator.

5. For an approximately Gaussian distribution, a nominal 95% interval is predictive mean ± 1.96 × predictive standard deviation; test empirical coverage.

6. If outputs are normalised, inverse-transform predictions/intervals correctly. For nonlinear transformations, transforming interval endpoints and recomputing variance are not interchangeable.

Deliverables: MAE/RMSE; coverage at 50%, 80%, 90%, 95%; mean interval width; calibration plots; results by dataset/horizon; aleatoric, epistemic and total variance comparisons.

Acceptance test: Intervals used in evaluation follow the same uncertainty definition claimed in the manuscript.

6. Verify graph and diffusion implementation

Priority: Essential
Why: Graph direction, transition-matrix convention and diffusion order must match the equations and implementation.

Files to inspect: Graph construction, adjacency/transition matrix creation and diffusion component.

1. Define whether A_ij means an edge from sensor i to sensor j.

2. Verify row/column convention of transition matrices and feature aggregation.

3. Check whether the graph is directed or symmetrised; verify forward/reverse diffusion matrices.

4. Confirm whether the implementation uses one-hop propagation or multiple powers of the transition matrix.

5. Record node count, directed edge count, self-loops and distance threshold for each dataset.

6. Validate graph construction against the actual sensor adjacency files.

Deliverables: Graph statistics for both datasets, documented adjacency convention, a small hand-constructed graph unit test and equations consistent with code.

Special check: diffusion ablation

• The reported normalised MAE increase from 0.4392 to 23.617 when diffusion is removed is unusually large and must be verified.

• Use identical splits, scaling, seeds, training budget and evaluation code for full and no-diffusion variants.

• Check output scale/shape, exploding predictions, invalid values and inconsistent loss reductions.

• Recompute metrics independently from saved predictions and repeat across seeds.

• Do not discard the result simply because it is extreme; establish whether it is reproducible.

7. Re-run core experiments systematically

Priority: Essential after code fixes
Why: Old results are provisional if the pipeline or architecture changes.

Files to inspect: Experiment configuration, training scripts, checkpointing and metric scripts.

1. Run corrected model on PEMS-BAY and METR-LA.

2. Use at least five random seeds for final model results and key ablations where feasible; if fewer, report the exact number and treat results as preliminary.

3. Save each run's seed, configuration, best validation epoch, checkpoint, logs, predictions and metrics.

4. Report mean and standard deviation across runs and horizon-specific results.

5. Run final test evaluation only after selecting configuration using validation data.

Deliverables: Reproducible experiment folder and tables reporting mean ± standard deviation.

Core comparisons to retain

• Proposed UA-GNN.

• Proposed model without diffusion.

• Proposed model without MC dropout.

• Proposed model without aleatoric variance, if this ablation is well-defined.

• Persistence/last-observation baseline.

• Relevant published baselines, labelled as reproduced or literature-reported.

Fair comparison: Published values are not matched experiments unless the baseline uses the same split, preprocessing and metric definitions. Label each result accordingly.

8. Add realistic sensor-failure experiments

Priority: Strongly recommended
Why: Independent random dropout alone does not cover practical failure patterns.

Files to inspect: Robustness evaluation script and corruption-generation utilities.

1. Evaluate clean input, random sensor dropout, persistent sensor failure, spatially clustered failure and temporally contiguous missing blocks.

2. Start with 10%, 20% and 30% corruption rates if feasible. Define whether the rate applies to nodes, sensor-time observations or whole windows.

3. Apply the same corrupted input to every compared model; keep targets and evaluation masks consistent.

4. Report MAE/RMSE and degradation relative to clean input; repeat across corruption seeds.

5. Separate original missingness from artificial test-time corruption and include a simple imputation baseline if missing-input handling is claimed.

Deliverables: Tables and plots of error versus corruption type/rate with repeated-seed summaries.

9. Additional metrics that can strengthen the paper

Metric / analysis	What to report	Why it matters
Forecast accuracy Essential	MAE and RMSE per horizon and dataset; mean ± SD across seeds.	Shows degradation as forecast horizon increases.
Calibration Essential	Coverage at 50%, 80%, 90%, 95%; mean interval width; nominal-vs-empirical plot.	Checks reliability and whether intervals are excessively wide.
Probabilistic quality Strongly recommended	CRPS when a defensible predictive distribution is available; otherwise interval score or weighted interval score.	Proper scores evaluate both accuracy and uncertainty; coverage alone is insufficient.
Robustness Strongly recommended	Error degradation versus clean input for random, persistent, spatial and temporal-block failures.	Quantifies performance under realistic sensor failures.
Efficiency Recommended	Parameter count, inference latency, peak memory, throughput; state hardware, batch size and method.	Supports deployment claims better than parameter count alone.
Statistical reliability Recommended	Confidence intervals across runs and paired comparisons on same test windows where appropriate; account for multiple comparisons.	Helps judge whether differences are robust to randomness.

Metric caution: Avoid MAPE as a headline metric without justification because zero or near-zero speeds can make percentage errors unstable. Do not let one ECE number replace coverage plots and interval-width measures.

10. Suggested division of work between two students

Student 1 — Data pipeline and model correctness

• Fix chronological splitting, train-only scaler/imputer fitting and causal missing-value handling.

• Verify tensor shapes and correct temporal convolution/LSTM axes if needed.

• Audit graph construction and diffusion direction.

• Correct dropout/batch-normalisation behaviour during MC sampling.

• Add tests for split boundaries, tensor dimensions and uncertainty output shapes.

• **Deliver:** corrected code, change log, passing tests and one documented smoke-test run.

Student 2 — Evaluation, calibration and experiments

• Correct uncertainty aggregation and prediction-interval calculations.

• Use validation-only checkpoint selection and prevent repeated test-set inspection.

• Implement metric calculations and horizon-wise reporting.

• Prepare repeated-seed runs and diffusion/uncertainty ablations.

• Prepare clean-input and sensor-failure evaluations.

• **Deliver:** evaluation scripts, saved predictions, metric tables, calibration plots and experiment logs.

Joint responsibility: Agree on tensor conventions, split boundaries and metric definitions; review each other's changes; run final experiments from the same corrected codebase; make every table and figure reproducible from saved outputs.

Dependency: If architecture changes affect outputs, Student 2 should wait until Student 1's implementation passes shape and correctness tests.

11. Final checklist before accepting results

Data and model correctness

■ Preprocessing fitted on training data only

■ No future-based imputation in forecasting inputs

■ Chronological split and window boundaries verified

■ Temporal operations act on the time axis

■ Graph orientation and diffusion verified

- MC dropout tested without updating batch-normalisation statistics

Evaluation

- Validation-only checkpoint selection
- Test set evaluated only after decisions are final
- Aleatoric and epistemic variance combined correctly
- Prediction intervals and calibration metrics verified
- Diffusion ablation reproduced and checked
- At least five seeds for final core results, or limitation documented

Experimental evidence

- Results separated by dataset and horizon
- Clean and sensor-failure results saved
- Baselines labelled as reproduced or literature-reported
- Mean and standard deviation reported
- Inference latency and memory measured if deployment is claimed

Paper consistency

- Tables and plots regenerated from corrected outputs
- All manuscript numbers updated
- Architecture diagram and equations match code
- Limitations and experimental protocol documented

Recommended order of work

1. Correct data preprocessing and architecture.

2. Verify graph operations and uncertainty calculations.

3. Run one end-to-end experiment and inspect outputs.

4. Run multi-seed experiments, ablations and robustness tests.

5. Regenerate paper tables and figures from corrected outputs.

Final recommendation

Do not spend the full experimental budget repeating the current implementation before checking correctness. If correcting the temporal architecture materially changes the model, the core results, methodology and conclusions may need to change. Report corrected evidence transparently, even if it is less favourable.

Scope note: This is a practical plan based on the preceding source-code review. Suspected issues should be confirmed against actual code and tests before being treated as established implementation failures. This plan does not claim that experiments have already been rerun.